

Chapitre 7 – Triggers – Contraintes de domaine

I.	Objectif	1
II.	Contrôle de cohérence dans l'enregistrement	1
III.	Contrôle de cohérence entre enregistrements	2

I. Objectif

Le trigger permet d'effectuer un contrôle avant/après une modification sur une table (update, insert ou delete), et le cas échéant d'annuler l'opération.

C'est le meilleur moyen pour appliquer des règles de cohérences, car elles sont au plus près des données (appliquées par le SGBD), donc sûres, par rapport à du code habituel. Dans les exemples, on utilisera la table *periode* (***per_no***, *per_debut*, *per_fin*, ...).

II. Contrôle de cohérence dans l'enregistrement

On souhaite s'assurer que deux champs dates indiquant une période sont valides (début antérieur à fin).

Utilisation avec MySQL

On définit un trigger **par opération** (update, insert ou delete). Ici, il faut en placer un sur insert et update. La requête créant le trigger contient des ; qui ne sont pas à interpréter comme séparateur. On modifie donc au début le séparateur de requête :

```
DELIMITER $$
```

```
CREATE TRIGGER periode_before_insert_ctrl_anteriorite
BEFORE INSERT ON periode FOR EACH ROW
BEGIN
    IF NEW.Per_Debut > NEW.Per_Fin
    THEN SIGNAL
        SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Date de début postérieure à date de fin';
    END IF;
END;

$$
```

On fait de même pour l'update, puis on rétablit le séparateur SQL :

```

CREATE TRIGGER periode_before_update_ctrl_anteriorite
BEFORE UPDATE ON periode FOR EACH ROW
BEGIN
    IF NEW.Per_Debut>NEW.Per_Fin
    THEN SIGNAL
        SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Date de début postérieure à date de fin';
    END IF;
END;

$$

DELIMITER ;

```

Les préfixes *NEW* et *OLD* permettent d'indiquer qu'on veut utiliser la valeur du champ après ou avant la modification. S'il s'agit d'un *INSERT*, seul *NEW* serait défini, de même pour *DELETE*, seul *OLD* sera présent.

La commande *SIGNAL* génère une exception qui permet de quitter le trigger mais aussi d'abandonner la-les requête-s en cours d'exécution, avec un code sans importance (*SQLSTATE*) et un message.

III. Contrôle de cohérence entre enregistrements

Il faut vérifier cette fois que les périodes ne se recouvrent pas d'un enregistrement à l'autre

Ceci est beaucoup plus intéressant, et ne peut être fait qu'avec les triggers. C'est un cas courant (prise de congé, formation, ...)

Périodes existantes	Début---Fin	Début---Fin
Nouvelle période		Début---Fin

La nouvelle période est **valide** car pour chaque période existante, on a $Fin < NEW.Début$ ou $Début > NEW.Fin$

Pour mettre en place le trigger, il suffit de compter les situations à problème (inverse de ci-dessus), et de refuser si ce décompte est > 0 :

```

DELIMITER $$

CREATE TRIGGER periode_before_insert_ctrl_non_recouvrement
BEFORE INSERT ON periode FOR EACH ROW
BEGIN
    IF (SELECT COUNT(*)
        FROM periode
        WHERE Per_Fin>=NEW.Per_Debut
        AND Per_Debut<=NEW.Per_Fin) > 0
    THEN SIGNAL
        SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Recouvrement avec une autre période';
    END IF;
END;

$$

```


Pour la mise à jour, il faut penser à ne pas comptabiliser la période qui va être modifiée :

```
CREATE TRIGGER periode_before_update_ctrl_non_recouvrement
BEFORE UPDATE ON periode FOR EACH ROW
BEGIN
    IF (SELECT COUNT(*)
        FROM periode
        WHERE Per_Fin>=NEW.Per_Debut
        AND Per_Debut<=NEW.Per_Fin
        AND Per_No<>NEW.Per_No) > 0
    THEN SIGNAL
        SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Recouvrement avec une autre période';
    END IF;
END;

$$

DELIMITER ;
```